Marine Plastic Tracker: Model, Data, and Trainin

Prepared from the current codebase on Thursday, July 30, 2026.

This note is written in simple words so we can explain the project clearly to the panel.

1. What the project does

Our project takes free Sentinel-2 satellite images of the sea, finds pixels that may contain floating material, uses AI to decide whether that material is plastic or not, groups nearby plastic pixels into hotspots, and predicts where those hotspots may drift over the next 72 hours.

Simple one-line answer:

"We built a system that looks at satellite images, finds possible floating waste, checks whether it is plastic, and shows where it may move next."

2. Which model is used?

The main deployed model is ‘marinenet.onnx‘.

It is a per-pixel neural network model. In simple words, it looks at one candidate pixel at a time and classifies it into one of 4 classes:

- Plastic
- Algae
- Sea_Foam
- Water

The current shipped model is a 3-seed ensemble of per-pixel MLPs. That means we trained the same kind of network multiple times and combined their outputs to make the result more stable.

There is also a second model called Clay. Clay is not the main detector. It is used as a second opinion verifier for hotspots after the main model has already detected possible plastic.

3. What is the model trained on?

The final deployed main model is trained on 2 real public benchmark datasets:

- MARIDA
- MADOS

These are real Sentinel-2 marine remote-sensing datasets with scientist-provided labels.

Important panel answer:

"Our final deployed model is trained on two real labeled satellite datasets: MARIDA and MADOS."

4. How many datasets are we working on?

There are 2 main datasets used for training the final model, and 2 more used for validation and extra testing.

Main training datasets

- MARIDA
- MADOS

Extra validation / generalization datasets

- PLP 2021 / 2022
- RefinedFloatingObjects (RFO)

So the clean answer is:

- 2 datasets for final model training
- 4 datasets in the wider research and validation workflow

5. Where did the datasets come from?

- MARIDA: Kikaki et al. 2022 marine debris benchmark
- MADOS: Kikaki, Kakogeorgiou et al. 2024 marine debris and oil-spill benchmark
- PLP: Plastic Litter Project, University of the Aegean
- RFO: RefinedFloatingObjects dataset by Russwurm et al. 2023

6. Where did we keep the datasets in the project?

We stored the downloaded raw datasets inside the ‘data/‘ folder:

- ‘data/marida‘
- ‘data/MADOS‘
- ‘data/plp/PLP‘
- ‘data/refinedfloatingobjects/refinedfloatingobjects‘

After cleaning and preprocessing, we saved compact training caches inside ‘python_engine/models/‘, such as:

- ‘marida_train.npz‘
- ‘mados_train.npz‘
- ‘marida_test.npz‘
- ‘mados_test.npz‘

Simple panel answer:

"We kept the raw datasets in the project’s data folder, then converted them into clean numpy cache files that the training scripts could use directly."

7. How much data trained the final model?

From the current cache files:

- MARIDA train: 347,204 labeled pixels
- MADOS train: 734,411 labeled pixels

Total training data for the final main model:

- 1,081,615 labeled pixels

Held-out test data:

- MARIDA test: 153,762 pixels
- MADOS test: 346,145 pixels

Important panel answer:

"Our final main model was trained on 1,081,615 labeled pixels from MARIDA and MADOS."

8. How did data preprocessing work?

We did not feed raw dataset files directly into the model. We first cleaned and converted them into the exact format needed by our project.

The preprocessing pipeline worked like this:

- Download the raw datasets and place them in the ‘data/‘ folder.
- Read the official train, validation, and test split files provided by each dataset.
- Open the satellite GeoTIFF bands and label masks using our own Python loaders.
- Select the Sentinel-2 bands that match our live pipeline.
- Compute extra spectral features such as FDI and NDVI.
- Convert the original dataset labels into our 4 project classes.
- Remove invalid, excluded, or non-finite rows.
- Save the clean result as ‘.npz‘ cache files.

9. What features were given to the model?

The current shipped model uses 11 features.

Raw bands

- B04

- B06
- B08
- B8A
- B11

Derived features

- FDI
- NDVI
- PI
- MNDWI
- NDVI_RE
- BRIGHT

These extra features are calculated from the same 5 satellite bands, so they improve the model without needing extra external data.

10. How did we manually implement preprocessing by ourselves?

We wrote our own dataset adapters and preprocessing code.

What we implemented manually:

- A loader for MARIDA
- A loader for MADOS
- A loader for PLP
- A loader for RFO
- Class mapping from original dataset labels to our 4 project classes
- Spectral feature calculation
- MADOS band upsampling to a common 10 m grid
- Cleaning and caching into ‘.npz‘ files

Simple panel answer:

"We did not just use a ready-made training pipeline. We wrote our own loaders, feature engineering, label mapping, cleaning, and dataset caching."

11. Which model did we train, and what does it do?

The most important trained model is ‘marinenet.onnx‘.

What it does:

- It takes the feature values of a candidate pixel.
- It predicts whether that pixel is Plastic, Algae, Sea_Foam, or Water.

There is also:

- ‘clay_head.pt‘: used for second-stage hotspot verification
- ‘marinenet_patch.onnx‘: an experimental patch CNN that we tested but did not keep as default

12. Full training process of the main model

This is the full training workflow in simple words:

- Load MARIDA and MADOS using our custom Python dataset loaders.
- Convert each labeled pixel into the model’s feature vector.
- Merge both datasets into one clean training set.
- Train the per-pixel MLP model on GPU.
- Check performance on held-out test data that the model never saw during training.
- Export the trained model to ONNX as ‘marinenet.onnx‘.
- Load that ONNX model into the live Python engine.

13. How does the model take data from the datasets?

There are 2 different stages to understand:

A. During training

- The loaders read the raw datasets from the ‘data/‘ folder.
- The preprocessing code converts them into clean feature arrays.
- The training script uses those clean arrays to train the model.

B. During live use

- The model does not go back to the old training datasets.
- Instead, the project downloads a new Sentinel-2 image.
- The pipeline computes the same kind of features from the new image.
- Then the trained ONNX model predicts the class of each candidate pixel.

This is an important panel point:

"The datasets are used to teach the model during training. During live use, the trained model works on new satellite images."

14. Full live pipeline after training

Once the model is trained and loaded, the project works like this:

- Download a real Sentinel-2 scene for the selected sea area.
- Mask land and cloud using the SCL layer.

- Compute FDI to find possible floating material.
- Build the same feature vector used during training.
- Run ‘marinenet.onnx‘ to classify each candidate.
- Keep the plastic detections.
- Group nearby plastic detections into hotspots using DBSCAN.
- Send hotspots to Clay for a second opinion.
- Run drift forecasting for 72 hours.
- Show the final result on the map and raise alerts if needed.

15. Results and accuracy

Current shipped main model results from ‘python_engine/models/metrics.json‘:

- Held-out accuracy: 98.99%
- Macro-F1: 0.8175
- Plastic precision: 0.7761
- Plastic recall: 0.8242
- Plastic F1: 0.7994

Per-dataset results:

- MARIDA held-out test: accuracy 0.9981, macro-F1 0.9047
- MADOS held-out test: accuracy 0.9862, macro-F1 0.7843

Important panel answer:

"The shipped main model reached 98.99% held-out accuracy and 0.8175 Macro-F1 on combined MARIDA and MADOS test data."

16. What about the other models?

Patch CNN

We also trained a patch CNN (‘marinenet_patch.onnx‘), but it performed worse:

- Test accuracy: 0.6236
- Macro-F1: 0.5548

So we did not keep it as the default model.

Clay verifier

Clay is used as a second-stage verifier, not the main detector.

Important current numbers:

- Clay held-out chip accuracy: 0.7624
- Clay Macro-F1: 0.7109
- Confirmed tier precision after fusion: 0.9219
- Confirmed tier recall after fusion: 0.7037

17. Independent validation results

We also checked the system using independent data outside the main training workflow.

PLP validation

Current saved validation result:

- 21 dates checked
- HDPE hit rate: 0.333
- FDI gate rate: 0.762

RFO holdout

This checks generalization on new seas:

- Clay debris recall: 0.5422
- Main per-pixel baseline on same holdout: 0.2222

This shows Clay is useful as a second opinion when the system sees new environments.

18. What did we manually implement by our own?

We manually implemented:

- Dataset integration
- Data preprocessing
- Feature engineering
- Label mapping
- Training scripts
- ONNX export
- ONNX model loading
- Live inference pipeline
- Hotspot clustering
- Clay second-stage verification
- Drift forecasting connection

Simple panel answer:

"We did not just download a ready-made model. We built the full pipeline ourselves: data loading, preprocessing, training, model export, and live integration."

19. Final 1-minute script for the panel

"Our project uses a main AI model called ‘marinenet.onnx‘. It is a per-pixel neural network that classifies candidate satellite pixels into Plastic, Algae, Sea_Foam, or Water.

We trained the final model on two real labeled benchmark datasets: MARIDA and MADOS. Together they gave us 1,081,615 labeled training pixels. We stored the raw datasets in the project’s data folder, wrote our own Python loaders, cleaned the data, mapped the original labels into our 4 project classes, calculated the spectral features, and saved clean cache files for training.

Then we trained the model on GPU, tested it on held-out data it had never seen before, and exported it to ONNX for live use in the project. The current shipped model got 98.99% accuracy and 0.8175 Macro-F1 on the combined held-out test.

In the live system, we download a Sentinel-2 image, compute FDI, run the trained model, group detections into hotspots, verify them with Clay, and finally forecast drift."

20. Very short answers to common questions

Which model is used?

- ‘marinenet.onnx‘

What is it trained on?

- MARIDA and MADOS

How many datasets are involved?

- 2 for final training, 4 in the wider workflow

How was preprocessing done?

- Band selection, feature engineering, label mapping, cleaning, and ‘.npz‘ caching

What were the final results?

- Accuracy 98.99%, Macro-F1 0.8175, Plastic F1 0.7994

What does Clay do?

- It gives a second opinion on hotspots